

UNIVERSIDAD POLITÉCNICA DE MADRID

E.T.S. DE INGENIERÍA DE SISTEMAS INFORMÁTICOS

PROYECTO FIN DE ASIGNATURA

GRADO EN INGENIERÍA DE COMPUTADORES

Programación de Hardware Reconfigurable

Implementación del procesador Intel 4004 en VHDL sobre FPGA Basys 3 Artix-7

Desarrollado por: Ander Regidor Barrante
Iván Serrano Prieto
María José García-Prieto Castells
Elena Esparza Baña

Dirigido por: Vicente A. García Alcántara

Madrid, 14 de mayo de 2026



Implementación del procesador Intel 4004 en VHDL sobre FPGA Basys 3 Artix-7

Desarrollado por: Ander Regidor Barrante

Iván Serrano Prieto

Maria José García-Prieto Castells

Elena Esparza Baña

Dirigido por: Vicente A. García Alcántara

Proyecto Fin de Asignatura, 14 de mayo de 2026

E.T.S. de Ingeniería de Sistemas Informáticos

Campus Sur UPM, Carretera de Valencia (A-3), km. 7

28031, Madrid, España

Si deseas citar este trabajo, la entrada completa en BibTeX es la siguiente:

```
@mastersthesis{2026AnderRegidorBarranteIvnSerranoPrietoMariaJosGarcaPrietoCastellsElenaE
  title = {Implementación del procesador Intel 4004 en VHDL sobre FPGA Basys 3
Artix-7},
  type = {Bachelor's Thesis},
  author = {Regidor Barrante, A.; Serrano Prieto, I.; García-Prieto Castells,
M. J.; Esparza Baña, E.},
  school = {E.T.S. de Ingeniería de Sistemas Informáticos},
  year = {2026},
  month = {5},
}
```

Esta obra está bajo una licencia [Creative Commons «Atribución-NoComercial-CompartirIgual 4.0 Internacional»](https://creativecommons.org/licenses/by-nc-sa/4.0/). Obra derivada de <https://github.com/blazaid/UPM-Report-Template>.



Todo cambio respecto a la obra original es responsabilidad exclusiva del presente autor.

Resumen

Este trabajo recoge el diseño e implementación en [VHDL](#) del procesador [CPU Intel 4004](#), el primer microprocesador comercial de la historia, sobre una placa de desarrollo Basys 3 equipada con una [FPGA Artix-7](#) de Xilinx.

El diseño sigue fielmente la especificación técnica original del Intel 4004, organizando el hardware en bloques funcionales equivalentes a los del diagrama oficial: ruta de datos, unidad de control, pila de direcciones, banco de registros y lógica de interfaz con memoria externa. Cada bloque se describe en esta memoria junto con las decisiones de implementación que lo justifican.

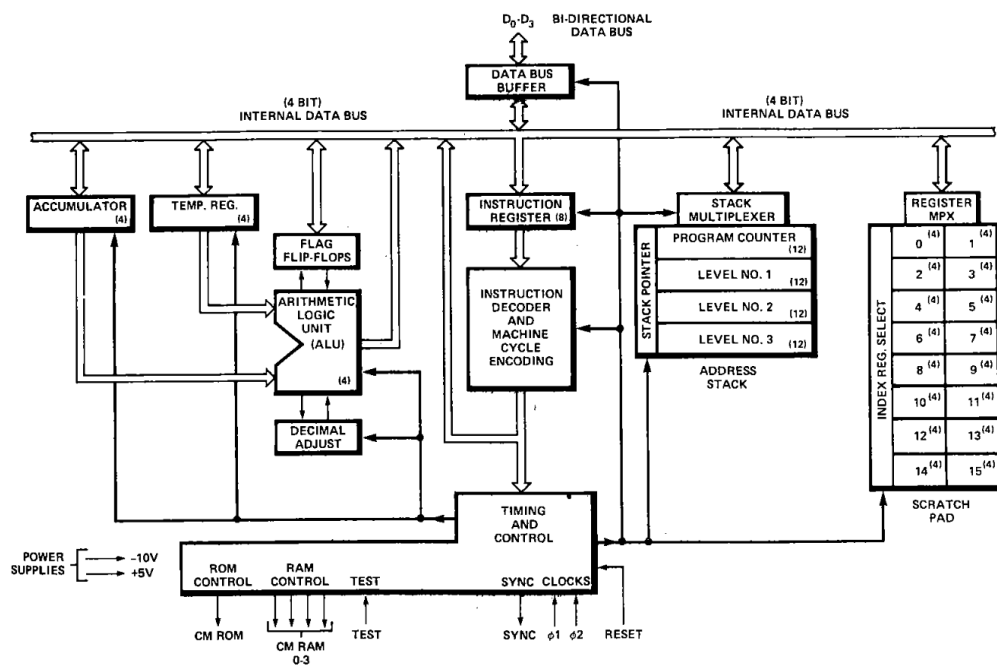


Diagrama de bloques del procesador Intel 4004

Palabras clave: VHDL; Intel 4004; FPGA; Artix-7; Arquitectura de computadores

Índice general

1	Introducción	1
1.1	Objetivos	1
1.2	Estructura de la memoria	2
2	Estado de la cuestión	3
2.1	El procesador Intel 4004	3
2.2	Herramientas utilizadas	3
3	Metodología	4
4	Diseño	5
4.1	Estructura del proyecto en código	5
4.2	Primitivas de hardware	7
4.3	Ruta de datos	9
4.4	Memoria y direccionamiento	12
4.5	Interfaz con el exterior	15
4.6	Unidad de control	17
4.7	Bloques pendientes	19
5	Verificación	20
6	Implementación	21

Índice general	II
7 Conclusiones	22
A Listado de ficheros del proyecto	23
Glosario	24
Siglas	24

Índice de tablas

4.1	Puertos de la entidad <code>d_ff_en</code>	7
4.2	Puertos del <code>registro_4b</code> (análogos en <code>registro_12b</code>)	8
4.3	Puertos de la entidad <code>flipflopJK</code>	8
4.4	Puertos de la entidad <code>accumulator</code>	9
4.5	Puertos de la entidad <code>alu</code>	10
4.6	Operaciones de la ALU	10
4.7	Puertos de la entidad <code>decimal_adjust</code>	11
4.8	Puertos de la entidad <code>flag_flip_flops</code>	12
4.9	Puertos de la entidad <code>scratch_pad</code>	13
4.10	Puertos de la entidad <code>stack_3L</code>	14
4.11	Entradas de datos al <code>bus_interno_mux</code> (cada una acompañada de su señal <code>_oe</code>)	16
4.12	Puertos de la entidad <code>data_bus_buffer</code>	16
4.13	Puertos de la entidad <code>instruction_register</code>	17
4.14	Fases del ciclo de máquina del Intel 4004 y su función	18
A.1	Ficheros fuente VHDL del proyecto	23

Índice de listados

4.1	Contenido del paquete <code>pkg_4004</code>	6
4.2	Serialización del stack en <code>mtx_stack</code>	15

1.

Introducción

El Intel 4004, lanzado en 1971, fue el primer [Unidad Central de Proceso \(CPU\)](#) comercial integrado en un único circuito integrado. Con una arquitectura de 4 bits, 700 kHz de frecuencia máxima y apenas 2300 transistores, supuso el punto de partida de la industria del microprocesador.

El objetivo de este proyecto, enmarcado en la asignatura [Programación de Hardware Reconfigurable \(PHR\)](#) del Grado en Ingeniería de Computadores de la *Escuela Técnica Superior de Ingeniería de Sistemas Informáticos* de la *Universidad Politécnica de Madrid*, es implementar fielmente este procesador en [VHDL](#) y sintetizarlo sobre una [FPGA](#) Artix-7 Basys 3.

1.1. Objetivos

El objetivo principal es implementar el procesador Intel 4004 de forma funcional en [VHDL](#), verificar su correcto funcionamiento mediante simulación y programarlo sobre la plataforma hardware Basys 3. Se establecen los siguientes objetivos específicos:

1. Estudiar y comprender la especificación técnica original del Intel 4004.
2. Diseñar la arquitectura en [VHDL](#) dividida en bloques funcionales fiel al diagrama de la especificación.
3. Verificar cada bloque de forma individual mediante *testbenches*.
4. Sintetizar e implementar el diseño completo sobre la Basys 3 Artix-7.
5. Documentar todas las decisiones de diseño en esta memoria.

1.2. Estructura de la memoria

El capítulo 2 describe brevemente la arquitectura del Intel 4004 y el marco tecnológico del proyecto. El capítulo 3 presenta la metodología de diseño seguida. El capítulo 4 contiene el grueso técnico del trabajo: la estructura del código VHDL y la descripción razonada de cada bloque. Los capítulos sucesivos, que se irán completando conforme avance el proyecto, recogerán la verificación, la implementación sobre FPGA y las conclusiones.

2. Estado de la cuestión

2.1. El procesador Intel 4004

El Intel 4004 fue diseñado por Federico Faggin, Marcian Hoff, Stanley Mazor y Masatoshi Shima para la calculadora Busicom. Su arquitectura define un conjunto de instrucciones de 46 operaciones con palabras de instrucción de 8 bits (o 16 bits para instrucciones de dos ciclos), un bus de datos bidireccional de 4 bits y una frecuencia de operación de hasta 740 kHz.

Las características principales de su arquitectura son las siguientes: un bus de datos externo de 4 bits (D0–D3); un bus de direcciones multiplexado en el tiempo de 12 bits; un acumulador de 4 bits y su [ALU](#) asociada; un banco de 16 registros internos ([scratch-pad](#)) de 4 bits organizables en 8 pares de 8 bits; una [pila de direcciones](#) de tres niveles de 12 bits para llamadas a subrutina; y lógica de control que genera las señales SYNC, CM-ROM y CM-RAM para la coordinación con la memoria externa.

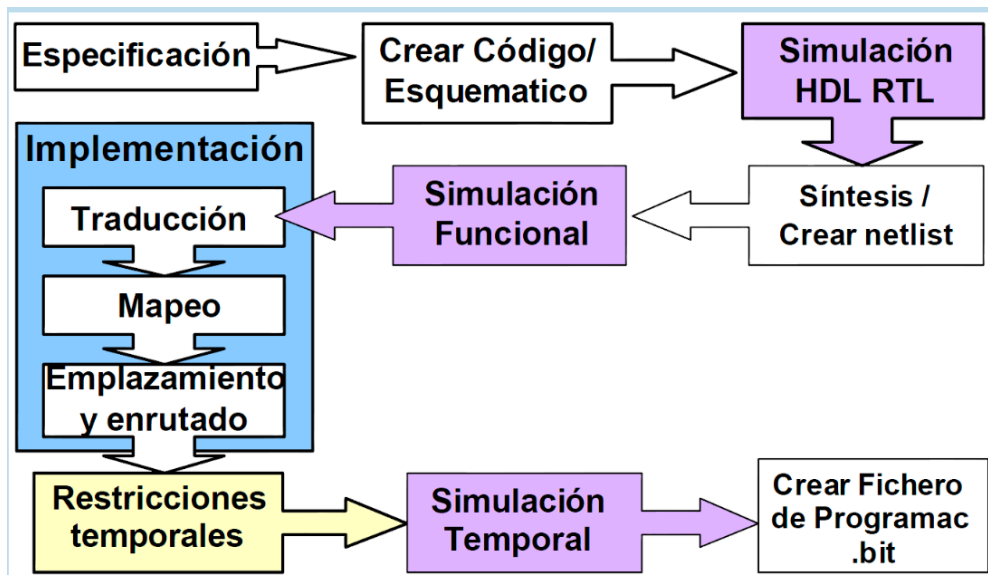
2.2. Herramientas utilizadas

El flujo de diseño empleado en el proyecto se apoya en las siguientes herramientas: *Vivado Design Suite* de Xilinx como entorno de síntesis e implementación; [VHDL-93](#) como lenguaje de descripción hardware; y la placa Basys 3 (Artix-7 XC7A35T) como destino hardware.

3.

Metodología

El proyecto sigue la metodología de diseño presentada en la asignatura [PHR](#), que estructura el trabajo en las siguientes fases: análisis de la especificación, diseño de la arquitectura [VHDL](#), verificación funcional mediante simulación, síntesis e implementación, y verificación sobre hardware. En el momento de redactar esta memoria el proyecto se encuentra en la fase de diseño.



Fases de la metodología de diseño de la asignatura [PHR](#)

Este capítulo describe en detalle el diseño [VHDL](#) del procesador Intel 4004. En primer lugar se expone la filosofía de organización del código; a continuación se describen, bloque a bloque, cada uno de los componentes que integran la [CPU](#).

4.1. Estructura del proyecto en código

4.1.1. Filosofía de diseño: arquitectura estructural jerárquica

Una de las primeras decisiones de diseño del proyecto fue adoptar una arquitectura puramente estructural en todos los niveles de la jerarquía. Esto significa que cada bloque funcional se implementa instanciando componentes de nivel inferior hasta llegar, en la base de la jerarquía, a primitivas de *flip-flop* individuales.

Esta decisión es intencionada y responde a varios criterios. Primero, permite que el mapeo de la herramienta de síntesis a recursos físicos de la [FPGA](#) sea completamente predecible: cada `registro_4b` se convierte en cuatro *flip-flops* `FDRE`, un bus de cuatro bits en cuatro rutas de señal, etcétera. Segundo, facilita la verificación incremental: cada primitiva puede simularse de forma aislada antes de integrarse en el bloque superior. Tercero, se mantiene una correspondencia directa entre el diagrama de bloques de la especificación original del Intel 4004 y la jerarquía de ficheros [VHDL](#).

4.1.2. Fichero raíz

Fichero: `cpu_4004_top.vhd`

El fichero `cpu_4004_top.vhd` es el punto de integración de todo el diseño. En él se declaran todas las señales internas del [bus interno](#) y de control, y se instancian todos los bloques funcionales mediante cláusulas `port map`. No contiene lógica funcional

propia: su única responsabilidad es ensamblar los componentes y cablear las señales entre ellos.

La entidad `cpu_4004_top` expone exactamente los pines físicos del chip Intel 4004 original en encapsulado DIP-16: los dos relojes en cuadratura (`clk_ph1`, `clk_ph2`), la señal de reset, el pin `test`, las salidas de control `sync`, `cm_rom` y `cm_ram`, y el bus de datos bidireccional de 4 bits `D_bus`.

4.1.3. Paquete de constantes

Fichero: `pkg_4004.vhd`

Para evitar el uso de literales numéricos dispersos por el código y facilitar posibles modificaciones futuras, se define el paquete `pkg_4004` que centraliza las constantes globales del diseño.

Listado 4.1. Contenido del paquete `pkg_4004`

```
constant BUS_W      : integer := 4;
constant BUS_CERO   : std_logic_vector(BUS_W-1 downto 0) := "
    0000";
constant BUS_ERROR   : std_logic_vector(BUS_W-1 downto 0) := "
    XXXX";
```

La constante `BUS_W` define el ancho del **bus interno**; todos los vectores de datos de 4 bits del diseño se dimensionan con ella. `BUS_CERO` representa el bus en estado inactivo y `BUS_ERROR` se utiliza para señalar colisiones de bus durante la simulación (véase la sección 4.5.1).

4.2. Primitivas de hardware

En la base de la jerarquía de diseño se encuentran tres primitivas que representan los elementos de memoria más simples.

4.2.1. *Flip-flop* D con enable

Fichero: `d_ff_en.vhd`

Es el elemento de memoria fundamental del diseño. Prácticamente todos los registros de la **CPU** se construyen como matrices de instancias de esta entidad. Su comportamiento es el de un *Flip-Flop* (FF) de tipo D síncrono con reset asíncrono activo alto y señal de habilitación:

Tabla 4.1. Puertos de la entidad `d_ff_en`

Puerto	Dir.	Bits	Descripción
<code>clk</code>	in	1	Reloj del sistema; captura en flanco de subida
<code>reset</code>	in	1	Reset asíncrono activo alto; pone <code>q</code> a '0'
<code>en</code>	in	1	Habilitación síncrona; si '0', el estado no cambia
<code>d</code>	in	1	Dato de entrada
<code>q</code>	out	1	Salida del FF

La arquitectura `Behavioral` implementa la lógica de forma directa: en el proceso sensible a `clk` y `reset`, si se produce `reset` la salida se lleva a '0' de forma asíncrona; en el flanco de subida del reloj, si `en` = '1', se captura el valor de `d`. La herramienta de síntesis mapea este patrón directamente sobre un primitivo `FDRE` de la Artix-7.

4.2.2. Registros de 4 y 12 bits

Fichero: `registro_4b.vhd`, `registro_12b.vhd`

Ambos registros se implementan de forma idéntica mediante una sentencia `GENERATE` que instancia N copias de `d_ff_en` en paralelo, siendo $N = 4$ para `registro_4b` y

$N = 12$ para `registro_12b`. Su interfaz es equivalente a la de `d_ff_en` salvo que los puertos de datos son vectores.

Esta estrategia de construcción mediante `GENERATE` es coherente con la filosofía estructural del proyecto: la descripción del registro no contiene ninguna inferencia de comportamiento; es literalmente la unión de N biestables individuales instanciados mediante síntesis paramétrica.

Tabla 4.2. Puertos del `registro_4b` (análogos en `registro_12b`)

Puerto	Dir.	Bits	Descripción
<code>clk</code>	in	1	Reloj del sistema
<code>R</code>	in	1	Reset asíncrono activo alto
<code>E</code>	in	1	Enable síncrono de escritura
<code>D_in</code>	in	4 (12)	Dato a cargar cuando <code>E = '1'</code>
<code>Q_out</code>	out	4 (12)	Dato almacenado

4.2.3. *Flip-flop JK*

Fichero: `flipflopJK.vhd`

Este **FF** de tipo JK se utiliza exclusivamente en el puntero de pila (véase la sección 4.4.2). Se implementa con arquitectura `FlujoDatos`: la salida complementada `notq` se obtiene como `not q_aux`, y la lógica JK se codifica como un proceso sensible al reloj y al reset.

Tabla 4.3. Puertos de la entidad `flipflopJK`

Puerto	Dir.	Bits	Descripción
<code>j</code>	in	1	Entrada J del biestable
<code>k</code>	in	1	Entrada K del biestable
<code>r</code>	in	1	Reset asíncrono activo alto
<code>clk</code>	in	1	Reloj de captura (flanco de subida)
<code>q</code>	out	1	Salida directa
<code>notq</code>	out	1	Salida complementada

4.3. Ruta de datos

4.3.1. Acumulador

Fichero: `accumulator.vhd`

El acumulador es el registro central de la [ALU](#): almacena el primer operando y el resultado de toda operación aritmética o lógica. Su implementación en [VHDL](#) es intencionalmente trivial: la entidad `accumulator` no es más que una envoltura (*wrapper*) sobre `registro_4b`. Esto es así porque el acumulador del Intel 4004 no tiene ninguna lógica especial asociada; su particularidad reside en el rol que juega en la microarquitectura, no en su comportamiento interno.

Tabla 4.4. Puertos de la entidad `accumulator`

Puerto	Dir.	Bits	Descripción
<code>clk</code>	in	1	Reloj del sistema
<code>reset</code>	in	1	Reset asíncrono
<code>load_en</code>	in	1	Señal de escritura procedente de <code>timing_and_control</code>
<code>d_in</code>	in	4	Dato a cargar (proviene del bus interno)
<code>q_out</code>	out	4	Contenido actual del acumulador (hacia la ALU y el bus)

El acumulador recibe el dato siempre del [bus interno](#) y vuelca su contenido directamente a la entrada A de la [ALU](#) y, cuando `acc_oe = '1'`, también al bus interno a través del multiplexor central.

4.3.2. Registro temporal

Fichero: `temp_register.vhd`

El registro temporal es estructuralmente idéntico al acumulador (igualmente un *wrapper* sobre `registro_4b`). Su función es almacenar el segundo operando de la [ALU](#) durante los ciclos de ejecución: en la fase X1, el controlador carga en él el valor del [scratchpad](#) que corresponda al registro fuente de la instrucción; en la fase X2, ese valor pasa a la entrada B de la [ALU](#).

La separación en dos entidades distintas (`accumulator` y `temp_register`) responde a razones de legibilidad y trazabilidad con la especificación, aunque su descripción VHDL sea idéntica.

4.3.3. Unidad Aritmético-Lógica

Fichero: `alu.vhd`

La **Unidad Aritmético-Lógica (ALU)** es el bloque combinacional que realiza todas las operaciones aritméticas y lógicas de la CPU. Su arquitectura `Combinational` no contiene ningún elemento de memoria; el resultado está disponible de forma continua en función de sus entradas.

Tabla 4.5. Puertos de la entidad `alu`

Puerto	Dir.	Bits	Descripción
<code>A_in</code>	in	4	Primer operando (salida del acumulador)
<code>B_in</code>	in	4	Segundo operando (<code>temp_register</code> o ajuste decimal)
<code>carry_in</code>	in	1	Acarreo de entrada (<code>flags_out[0]</code>)
<code>op</code>	in	3	Código de operación (véase la tabla 4.6)
<code>result_out</code>	out	4	Resultado de la operación
<code>carry_out</code>	out	1	Acarreo de salida
<code>zero_flag</code>	out	1	Activo si <code>result_out = 0000</code>

La **ALU** soporta siete operaciones seleccionadas mediante el vector de 3 bits `op`:

Tabla 4.6. Operaciones de la **ALU**

op	Operación	Descripción
000	ADD	$A + B + C_{in}$
001	SUB	$A + \overline{B} + C_{in}$ (resta en complemento a 1, fiel al 4004)
010	AND	$A \text{ AND } B$
011	OR	$A \text{ OR } B$
100	XOR	$A \text{ XOR } B$
101	PASS	Pasa A sin modificar (valor por defecto)
110	INC	$A + 1$

Internamente, los tres resultados de mayor coste (ADD, SUB, INC) se calculan siempre en paralelo mediante señales intermedias de 5 bits (`sum_result`, `sub_result`, `inc_result`), y un proceso `case` final selecciona cuál de ellos se propaga a la salida. La lógica para AND, OR y XOR y la operación PASS no requieren señales intermedias. Este enfoque, habitual en ALU combinacionales, garantiza que el camino crítico sea el de la suma y que el sintetizador pueda optimizar el árbol de acarreo.

La entrada B de la ALU proviene de un multiplexor en `cpu_4004_top`: normalmente conecta con `temp_register`, pero cuando `ctrl_alu_b_mux = '1'` conecta con la salida del bloque de ajuste decimal, permitiendo la instrucción DAA.

4.3.4. Ajuste decimal

Fichero: `decimal_adjust.vhd`

Este pequeño bloque combinacional calcula el valor de corrección BCD que se suma al acumulador para implementar la instrucción DAA (*Decimal Adjust Accumulator*). Si el acarreo está activo o el valor del acumulador supera 9 (es decir, no es un dígito BCD válido), el ajuste es 0110 (6 en decimal); en caso contrario, el ajuste es 0000 (sin corrección).

Tabla 4.7. Puertos de la entidad `decimal_adjust`

Puerto	Dir.	Bits	Descripción
<code>acc_in</code>	in	4	Contenido actual del acumulador
<code>carry_in</code>	in	1	Acarreo actual (<code>flags_out[0]</code>)
<code>adj_out</code>	out	4	Corrección a sumar (0000 o 0110)

Su salida `adj_out` se conecta en `cpu_4004_top` a la entrada B de la ALU a través del multiplexor seleccionado por `ctrl_alu_b_mux`.

4.3.5. Registros de flags

Fichero: `flag_flip_flops.vhd`

El bloque `flag_flip_flops` almacena el vector de estado de la CPU: el acarreo (`q_out[0]`), el flag de test (`q_out[1]`) y dos bits auxiliares (`q_out[2:3]`). Al igual que el acumulador, se implementa de forma estructural como cuatro instancias de `d_ff_en` generadas mediante `GENERATE`.

Tabla 4.8. Puertos de la entidad `flag_flip_flops`

Puerto	Dir.	Bits	Descripción
<code>clk</code>	in	1	Reloj del sistema
<code>reset</code>	in	1	Reset asíncrono
<code>load_en</code>	in	1	Señal de escritura desde <code>timing_and_control</code>
<code>d_in</code>	in	4	Dato a cargar (viene del bus interno)
<code>q_out</code>	out	4	Estado actual de los flags
<code>flags_oe</code>	out	1	OE hacia el bus interno (placeholder)

La señal `flags_oe` está actualmente fijada a '0' como *placeholder*: en la especificación, los flags pueden volcarse al bus en determinadas instrucciones, pero esta funcionalidad queda pendiente de implementación en el bloque de decodificación de instrucciones.

4.4. Memoria y direccionamiento

4.4.1. Banco de registros (*scratchpad*)

Fichero: `scratch_pad.vhd`, `mtx_scratchpad.vhd`

El *scratchpad* implementa los 16 registros de propósito general de 4 bits del Intel 4004 (R0–R15), que el programador puede agrupar en 8 pares de 8 bits (RR0, RR2, ..., RR14). La arquitectura Structural del bloque instancia 16 copias de `registro_4b` y un multiplexor 16:1 (`mtx_scratchpad`) para leer el registro seleccionado.

Tabla 4.9. Puertos de la entidad `scratch_pad`

Puerto	Dir.	Bits	Descripción
<code>clk_r</code>	in	1	Reloj del sistema
<code>rst_r</code>	in	1	Reset asíncrono
<code>w_e</code>	in	1	Write enable
<code>pair_mode</code>	in	1	'1': acceso a par de registros (8 bits); '0': acceso individual
<code>address</code>	in	4	Dirección del registro (0–15 individual, 0–7 par)
<code>data_in</code>	in	8	Dato a escribir (<code>[3:0]</code> en modo individual)
<code>data_out</code>	out	8	Dato leído (<code>[3:0]</code> en modo individual)

La lógica de habilitación se genera mediante un bucle `GENERATE` que produce 16 señales `en_regs(i)`. En modo individual, únicamente `en_regs(addr_int)` se activa. En modo par, se activan simultáneamente el registro par `reg_even` (que recibe `data_in[7:4]`) y el impar `reg_odd` (que recibe `data_in[3:0]`), modelando el acceso a los registros dobles `RRn` descritos en la especificación.

El multiplexor de lectura `mtx_scratchpad` selecciona siempre utilizando la dirección de 4 bits completa. En modo par, la salida `data_out` concatena `q_regs(reg_even)` & `q_regs(reg_odd)` para formar el byte `RRn` de 8 bits.

4.4.2. Pila de direcciones

Fichero: `stack_3L.vhd`

La [pila de direcciones](#) almacena las tres direcciones de retorno de subrutina que soporta el Intel 4004. Se implementa como cuatro registros de 12 bits (`registro_12b`) con un puntero de pila de 2 bits que selecciona cuál de ellos está activo en cada momento.

Tabla 4.10. Puertos de la entidad `stack_3L`

Puerto	Dir.	Bits	Descripción
<code>Clk_s</code>	in	1	Reloj del sistema
<code>U_D_s</code>	in	1	Dirección del puntero: '1' = push (subir), '0' = pop (bajar)
<code>D_12_s</code>	in	12	Dirección a escribir (nueva dirección de retorno)
<code>E_s</code>	in	1	Enable de escritura en el nivel activo
<code>R_s</code>	in	1	Reset global
<code>oe_s</code>	in	1	OE hacia el bus interno
<code>fase_s</code>	in	2	Fase del reloj para serialización ("00"=A1, "01"=A2, "10"=A3)
<code>sal_4_stck</code>	out	4	nibble serializado hacia el bus interno

Puntero de pila

Fichero: `puntero_stack.vhd`

El puntero de pila es un contador de 2 bits bidireccional construido con dos [FF](#) tipo JK (`flipflopJK`). La dirección de conteo (arriba o abajo) la controla la señal `up_down`: cuando es '1' el puntero avanza (push); cuando es '0' retrocede (pop). El incremento del segundo biestable se controla mediante el componente auxiliar `clk_up_down`, un multiplexor que elige entre `q1` y `not q1` como reloj del segundo [FF](#) según la dirección de conteo, implementando así un contador Johnson bidireccional.

Multiplexor de pila con serialización

Fichero: `mtx_stack.vhd`

El multiplexor de pila sirve un doble propósito: selecciona cuál de los cuatro registros de 12 bits es el activo según el puntero de pila, y serializa su contenido de 12 bits en tres **nibble** de 4 bits para volcarlo al **bus interno** a lo largo de las fases A1, A2 y A3 del ciclo de máquina. La serialización se implementa mediante un proceso secuencial (con registro, a diferencia del resto de multiplexores del diseño) cuya salida se actualiza en cada flanco de subida del reloj cuando `oe = '1'`:

Listado 4.2. Serialización del stack en `mtx_stack`

```
when "00" => y <= reg_activo(3 downto 0);    -- A1: nibble bajo
when "01" => y <= reg_activo(7 downto 4);    -- A2: nibble
        medio
when "10" => y <= reg_activo(11 downto 8);    -- A3: nibble alto
```

4.5. Interfaz con el exterior

4.5.1. Bus interno — multiplexor central

Fichero: `bus_interno_mux.vhd`

El **bus interno** de 4 bits es el canal de comunicación entre todos los bloques funcionales de la **CPU**. Una de las decisiones de diseño más significativas del proyecto es la forma en que se implementa este bus.

En **VHDL** lo más natural para modelar un bus compartido es el uso de señales de tipo `std_logic` con valor de alta impedancia 'Z' (*bus triestado*). Sin embargo, este proyecto opta deliberadamente por un multiplexor centralizado: el bloque `bus_interno_mux` recibe los 4 bits de salida de cada uno de los ocho emisores posibles, junto con sus respectivas señales `_oe`, y produce como salida el valor del único emisor que tiene permiso activo en cada instante.

Esta decisión tiene dos ventajas principales. Primero, los buses triestado internos sintetizan mal en **FPGA**: los recursos de la Artix-7 no tienen buffers triestado en la fabrica interna, por lo que el sintetizador los convierte igualmente en multiplexores, pero de forma implícita y menos predecible. Segundo, la detección de colisiones en simulación es trivial: si dos señales `_oe` se activan simultáneamente, el bus toma el valor `BUS_ERROR = "XXXX"`, que es inmediatamente visible en el visor de señales de Vivado.

Tabla 4.11. Entradas de datos al `bus_interno_mux` (cada una acompañada de su señal `_oe`)

Fuente	Descripción
<code>acc_out</code>	Acumulador
<code>tmp_reg_out</code>	Registro temporal
<code>flags_out</code>	Registros de flags
<code>alu_out</code>	Resultado de la ALU
<code>decoder_out</code>	Decodificador de instrucciones (pendiente)
<code>stack_out</code>	Pila de direcciones
<code>reg_mux_out</code>	<i>Scratchpad</i>
<code>ext_buf_out</code>	Buffer de bus de datos externo

4.5.2. Buffer de bus de datos

Fichero: `data_bus_buffer.vhd`

El `data_bus_buffer` es la interfaz entre el **bus interno** y el pin bidireccional externo `D_bus` del chip. Cuando `dir_out = '1'`, el bus interno se vuelca al exterior mediante un condicional triestado; cuando `dir_out = '0'`, el pin externo alimenta el bus interno (`internal_bus_out <= data_bus_ext`).

Tabla 4.12. Puertos de la entidad `data_bus_buffer`

Puerto	Dir.	Bits	Descripción
<code>internal_bus_in</code>	in	4	Dato procedente del bus interno
<code>internal_bus_out</code>	out	4	Dato hacia el bus interno (lectura del pin externo)
<code>dir_out</code>	in	1	'1': la CPU escribe en el bus externo
<code>data_bus_ext</code>	inout	4	Pin bidireccional <code>D0--D3</code> del Intel 4004

A diferencia del bus interno, aquí el triestado es correcto y necesario: `data_bus_ext` es un pin físico del encapsulado del chip, y los pines `I0B` de la Artix-7 sí disponen de buffers triestado reales.

4.6. Unidad de control

4.6.1. Registro de instrucción

Fichero: `instruction_register.vhd`

El **Registro de Instrucción (IR)** captura la instrucción de dos **nibble** desde el **bus interno** a lo largo de dos fases consecutivas del ciclo de búsqueda. Se implementa de forma estructural como ocho instancias de `d_ff_en` organizadas en dos grupos de cuatro mediante `GENERATE`: el grupo alto (*high nibble*, bits 7–4) se captura en la fase M1 cuando `load_high = '1'`; el grupo bajo (*low nibble*, bits 3–0) se captura en la fase M2 cuando `load_low = '1'`.

Tabla 4.13. Puertos de la entidad `instruction_register`

Puerto	Dir.	Bits	Descripción
<code>clk</code>	in	1	Reloj del sistema
<code>reset</code>	in	1	Reset asíncrono
<code>load_high</code>	in	1	Carga el nibble alto (activo en M1)
<code>load_low</code>	in	1	Carga el nibble bajo (activo en M2)
<code>bus_in</code>	in	4	Dato del bus interno
<code>instr_out</code>	out	8	Instrucción completa (8 bits) hacia el decodificador
<code>decoder_out</code>	out	4	Nibble bajo (placeholder para conexión al decodificador)
<code>decoder_oe</code>	out	1	OE del decodificador (placeholder, fijo a '0')

Los puertos `decoder_out` y `decoder_oe` son *placeholders* a la espera de que se implemente el bloque `instruction_decoder`. En el `cpu_4004_top` actual, ambas salidas del **IR** se conectan a open.

4.6.2. Temporización y control

Fichero: `timing_and_control.vhd`

El bloque `timing_and_control` es el corazón de la unidad de control. Implementa la *Finite State Machine* (FSM) de 8 estados que corresponde al ciclo de máquina del Intel 4004 (A1, A2, A3, M1, M2, X1, X2, X3) y genera todas las señales de control del *datapath*.

La FSM se describe con arquitectura `Behavioral`: un proceso secuencial sensible a `clk_ph1` y `reset` incrementa un contador de 3 bits en cada flanco de subida, ciclando de 000 a 111 y volviendo a 000. Un segundo proceso combinacional, sensible al estado y a las entradas de control, produce las señales de salida mediante una sentencia `case`.

Tabla 4.14. Fases del ciclo de máquina del Intel 4004 y su función

Estado	Fase	Función
000	A1	Inicio del ciclo; <code>sync = '1'</code> ; nibble bajo de la dirección al bus externo
001	A2	Nibble medio de la dirección al bus externo
010	A3	Nibble alto de la dirección al bus externo
011	M1	Lectura del nibble alto de la instrucción desde memoria
100	M2	Lectura del nibble bajo de la instrucción desde memoria
101	X1	Primera fase de ejecución (carga de operando en temp. register)
110	X2	Segunda fase de ejecución (operación ALU, escritura en acumulador)
111	X3	Tercera fase de ejecución (saltos, escritura en RAM, etc.)

Las señales de entrada `inst_group` (vector *one-hot* de 16 bits que indica el tipo de instrucción) y `current_frag` (que distingue instrucciones de uno o dos ciclos) provienen del bloque `instruction_decoder`, aún pendiente de implementación. Hasta entonces, el controlador contiene código de ejemplo para la instrucción ADD (`inst_group(8) = '1'`) como plantilla para el resto de instrucciones.

4.7. Bloques pendientes

A fecha de redacción de este capítulo, el diseño cuenta con dos bloques aún no implementados:

`instruction_decoder.vhd` Debe traducir la instrucción de 8 bits almacenada en el `IR` a un vector *one-hot* (`inst_group`) y la señal `current_frag` para el bloque de temporización. Su conexión en `cpu_4004_top` está reservada pero comentada.

ROM Memoria de programa externa a la `CPU` que proporciona los bytes de instrucción durante las fases M1 y M2. Se contempla su implementación como bloque `BRAM` de Vivado o ROM distribuida.

5.

Verificación

6.

Implementación

7.

Conclusiones

A.

Listado de ficheros del proyecto

Tabla A.1. Ficheros fuente **VHDL** del proyecto

Fichero	Descripción
pkg_4004.vhd	Paquete global de constantes
cpu_4004_top.vhd	Integrador top-level
d_ff_en.vhd	FF D con enable (<i>primitiva base</i>)
flipflopJK.vhd	FF JK (usado en puntero de pila)
registro_4b.vhd	Registro de 4 bits
registro_12b.vhd	Registro de 12 bits
accumulator.vhd	Acumulador
temp_register.vhd	Registro temporal
alu.vhd	Unidad Aritmético-Lógica
decimal_adjust.vhd	Lógica de ajuste decimal BCD
flag_flip_flops.vhd	Registros de flags
scratch_pad.vhd	Banco de 16 registros (<i>scratchpad</i>)
mtx_scratchpad.vhd	Multiplexor 16:1 del <i>scratchpad</i>
stack_3L.vhd	Pila de direcciones de 3 niveles
puntero_stack.vhd	Puntero de pila bidireccional
mtx_stack.vhd	Multiplexor de pila con serialización
bus_interno_mux.vhd	Multiplexor central del bus interno
data_bus_buffer.vhd	Buffer triestado de bus externo
instruction_register.vhd	Registro de instrucción
timing_and_control.vhd	FSM de temporización y control
(pendiente)	instruction_decoder.vhd — Decodificador de instrucciones
(pendiente)	ROM de programa

Índice de términos

Glosario

bus interno Bus de datos de 4 bits que interconecta todos los bloques funcionales internos de la CPU. Equivale al bus *BUS* del diagrama de la especificación Intel 4004 [5](#), [6](#), [9](#), [12](#), [14–17](#)

nibble Grupo de 4 bits. La CPU Intel 4004 opera de forma nativa con nibbles al ser un procesador de 4 bits [14](#), [15](#), [17](#)

pila de direcciones Estructura de tres niveles de 12 bits cada uno que almacena las direcciones de retorno de las llamadas a subrutina [3](#), [14](#)

scratchpad Banco de 16 registros de 4 bits internos a la CPU, denominados R0–R15, que el programador puede usar como registros de propósito general [3](#), [9](#), [12](#)

Siglas

ALU Unidad Aritmético-Lógica [3](#), [9–11](#), [16](#), [18](#), [III](#)

BCD *Binary Coded Decimal* [11](#), [23](#)

BRAM *Block RAM* [19](#)

CPU Unidad Central de Proceso [1](#), [3](#), [5](#), [7](#), [10](#), [12](#), [15](#), [19](#)

FF *Flip-Flop* [7](#), [8](#), [14](#), [23](#)

FPGA *Field Programmable Gate Array* [1–3](#), [5](#), [16](#)

FSM *Finite State Machine* [18](#), [23](#)

IR Registro de Instrucción [17](#), [19](#)

OE *Output Enable* [12](#), [14](#), [17](#)

PHR Programación de Hardware Reconfigurable [1](#), [4](#)

VHDL *Very High Speed Integrated Circuit Hardware Description Language* [1–5](#), [9](#), [10](#), [15](#),
[23](#), [III](#)

